

物件導向程式設計
呂東原 助理教授

Outline

- 上課規定
- 評分方式與上課內容
- 作業與小專題要求
 - 計畫書(小專題)
 - 流程圖與方塊圖
 - 命名原則
- .Net 與Framework的關係
 - C#的特性與應用

電腦教室規定

- 請勿干擾同學與教學進行
- **禁止攜帶飲食**：實驗室內**嚴禁飲食、喝水、儲存食物**，避免液體潑濺引發電路短路。
- **嚴禁追逐嬉戲**：禁止在實驗室內跑步或進行與實驗無關的活動。
- **維持桌面整潔**：實驗桌上不得放置飲料或無關的雜物，油類或水若傾倒應立即清除
- 本課程“必定點名”，第一節和第三節都會點。

評分方式

- 出席率與學習態度
 - 20%
- 討論與報告
 - 這裡指的是課時練習後的完成度，與一般回家作業
 - 30%
- 期中考
 - 20%
- 期末報告
 - 題目需要自定，再經老師核准才算數
 - 若與專題相關，**務必先問過專題老師**意願!!!! (學術倫理)
 - 30%



程式設計－使用 Visual C# 2022 (附範例光碟)

全華圖書, 周家揚 0958-008-937



課程內容

- 期中以前:基礎程式與基本物件導向程式
以TQC+物件導向程式設計#7 前三題為標準
額外引入LINQ語法(C#)
- 期中到期末:物件導向程式與視窗程式設計
結合物件導向與視窗程式應用
- 會依照教學狀況調整內容與進度
 - 過去同學們應該已經學過C++物件導向程式
 - 可能會加快進度(如果同學能快速的學習課程內容)，或於每節課第三堂加些教科書內較缺少，較有趣的內容
- 額外內容(視狀況而定)
 - 其他程式與C#的互相連結
 - 第三方程式/函式庫的應用
 - ML.Net的使用

期中與期末成績計算

- 期中
 - 以TQC基礎物件導向程式語言C# 7, 前三題為考題
 - 同學亦可視狀況參與TQC+考式
 - 基本處理
 - 條件判斷式、迴圈及例外處理
 - 陣列、集合與遞迴
- 期末
 - 以實作專題與報告型式(可連結其他第三方程式或周邊)
 - 要求內容:程式規劃書(想做什麼)、**功能方塊圖**、依照**命名原則**編寫程式、具備完整性的功能
 - 期中以前至少完成”程式規劃書”
 - 不要求多大型程式，但我希望”完整”(用於模擬開發專案的教學性)
- 額外加分標準
 - 上課期間，通過證照考試

獎勵學生參與證照考試

- 額外加分標準
 - 限定上課期間，通過證照考試
 - TQC+ 基礎物件導向程式語言 c# (C級)
 - TQC+ 物件導向視窗及資料庫程式設計 C# (B級)
 - TQC+ 物件導向程式語言 C# (但這門目前沒考試試場)
 - 電腦軟體設計乙級 (C++)
 - CPE(限用C++)

會依進度修改

周次	主題內容/章節*
1	Visual C#程式語言、主控台應用程式與程式編寫要求
2	資料型態、變數與運算子、資料之輸入與輸出方法
3	中秋節
4	程式流程控制與流程圖
5	補假
6	內建類別

7 陣列與串列

8 例外處理

9 期中

10 自訂類別

11 繼承

12 視窗應用程式

13

視窗常用控制項

14

滑鼠事件與鍵盤事件

15

交談式控制項與檔案處理、期末專題

16

行憲紀念日

17

元旦

18

期末專題

計畫書(小專題)

軟體開發規劃書

software project management plan

Why-Who-What-How

這個專案是什麼？

要解決/處理什麼事情

MVP (最小可行產品) 是什麼？

必要與核心功能性

有哪些是錦上添花的功能？

非必要但加分的功能性

這個專案什麼時候會完成？

沒有完成 等於 零成果，控制時程

企劃書與計畫書的差別

專案企劃書		專案計畫書
要求投資為導向		如何完成為導向
專案啟動前提案階段	適用階段	核准後規劃執行階段
說服決策者核准專案	核心目的	指導團隊實際執行
為什麼做、做什麼	回答問題	怎麼做、何時完成
高階主管、投資人	主要讀者	專案團隊、執行單位

計畫書簡介

實際上，應該依照各行業需求而再修訂

- 專案計畫書（Project Plan）
 - 目的：定義專案目標、範疇、時程和資源，為專案提供藍圖
- 大綱
 - 專案背景與目的：簡述專案的需求、問題、目標和價值。（目標）
 - 專案範疇：明確專案範疇，包括要完成的主要項目及排除在外的部分，避免後期範疇擴張。（產品功能範圍）
 - 目標和可交付成果：列出具體目標和交付物，確保專案成果明確可測量。（驗收目標）
 - 工作分解結構（WBS）：將專案分解為主要任務和子任務，以清晰地展示工作分配。（工作方塊圖拆解細分）

計畫書簡介

實際上，應該依照各行業需求而再修訂

- 大綱
 - 資源需求：包括人力、設備、軟硬體、經費等需求。(資源需求，可省略)
 - 時程安排：包含主要的專案階段和時間表。(時程圖/甘特圖)
 - 風險分析與應對措施：列出可能的風險和相應的風險管理計劃。(風險，可省略)
 - 溝通計劃：詳細說明項目報告、會議、通訊工具和頻率(橫向/縱向溝通，可省略)(多人協作同一計畫，會要求!!!!)
 - 專案結束條件：定義何時視為專案完成以及評估標準。(目標完成與評估)

區塊圖 vs 流程圖：核心差異比較

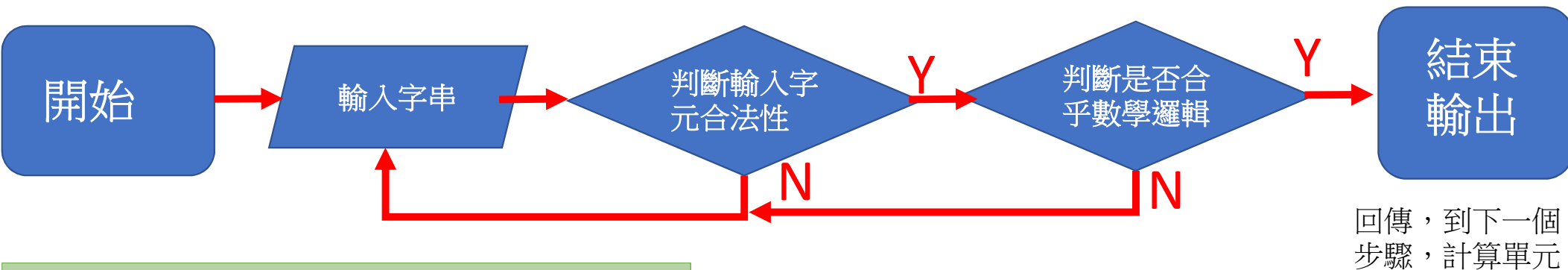
	區塊圖 (Block Diagram)	流程圖 (Flowchart)
主要目的	展示系統的硬體或功能架構組成	展示演算法或操作過程的步驟與時序
核心焦點	結構 (Structure) 與組件關聯	行為 (Behavior) 與條件邏輯
圖形符號	主要使用簡單方塊與單/雙向箭頭。	使用圓形、矩形、菱形等多種標準化符號。
時間與順序概念	通常無明確時間順序，側重資料/訊號流向	具有嚴格的時間順序與執行流向程式邏輯、操作步驟、決策分析（從上到下/左到右）
層次感	偏向層級化、分組結構 (Hierarchical)。	偏向線性、分支路徑 (Sequential/Logic-based)。
適用領域	系統架構設計、硬體配置、組織架構圖。	業務流程、程式邏輯、操作步驟、決策分析。

例，計算機為簡例

方塊圖



流程圖:以輸入單元為簡例



程式實現上方流程

計畫書要求 - 小結

- 要有一個“故事”，即要處理解決的問題(即使已被解決也沒問題，這只是訓練)
- 計畫書:方塊圖，主要描述每個結構上，要做什麼的“架構”
- 程式流程:流程圖，主要描述怎麼做到要求的邏輯與決策方法
- 程式碼要求，依照此課程的命名原則(簡略版)
- 所以建議大家習慣在寫程式時先想好架構圖與流程圖，在大型程式架構下才能拆分出各子項功能，並實現之。

計畫書 – 範例



- 先建網址，未來我寫完再上傳
- 之後如果我要上傳資料/程式碼，皆在這個位置
- <https://github.com/DongYuan-Lyu/Object-Oriented-Programming>
- 經驗提醒：
- 隨時做好備份(不管是用線上/自己備份/任何方法)
 - 防止改到後來亂掉
 - 防止硬碟故障
 - 最重要的，防止C#程式本身造成的異常，如更新版本、不小心修改到UI/程式設定資料

命名原則
本課程要求在四頁後，
其他是介紹

命名規則 - C# 識別碼命名規則和慣例

此頁會依各公司/實驗室/使用程式而改變

- 命名請用“有意義”的文字，勿用太常見的文字，如age (x，太常見，無法一眼看出表達意義)；studentAge(o，區域變數小駝峰式)

程式元件 (Element)	命名風格 (Style)	範例 (Example)	備註與規範
類別 (Class)	PascalCase	CustomerService	名詞或名詞短語
結構 (Struct)	PascalCase	CustomPoint	名詞或名詞短語
介面 (Interface)	PascalCase	IUserRepository	強制以 I 前綴開頭
列舉類型 (Enum)	PascalCase	UserStatus	一般用單數；加 [Flags] 時用複數
屬性 (Property)	PascalCase	TotalAmount, IsActive	布林屬性常以 Is / Has / Can 開頭
方法 (Method)	PascalCase	CalculateDiscount()	動詞或動詞短語

程式元件 (Element)	命名風格 (Style)	範例 (Example)	備註與規範
事件 (Event)	PascalCase	OrderCompleted	使用動詞過去分詞或現在進行式
私有欄位 (Private Field)	_camelCase	_dbContext, _retryCount	前綴加上底線 _
區域變數 (Local Variable)	camelCase	itemCount	不可使用底線開頭
方法參數 (Parameter)	camelCase	orderId	需具備明確意義 .NET 官方推薦
常數 (Constant)	PascalCase	MaxLimit	PascalCase，而非全大寫
泛型型別參數 (Generic)	PascalCase	T, TEntity	以 T 開頭
非同步方法 (Async)	PascalCase	GetCustomerAsync()	後綴加上 Async
自訂例外 (Exception)	PascalCase	UserNotFoundException	後綴加上 Exception

續

- Local Function

- 官方命名規則與公用函式相同
- 團體自定規則有，開頭小寫，或是開頭使用_來表示之
- EX: CalculateTotal(官方)、calculateTotal、_calculateTotal (自訂)

- 私有欄位 開頭使用 _表示

- EX:

```
public class Person {  
    // 宣告私有欄位  
    private string _name;  
  
}
```

小結 命名規則

- 此小節為人為訂制規則，依賴團體協作要求，未來請依自己實驗室/公司設定為主
- 本課程的要求 (簡化版，未來請照協作單位要求)
 - 使用駝峰式
 - 常數使用"全大寫加_分格文字"(與官方不同)
 - 類別、屬性、方法、結構、事件，使用大駝峰式
 - 區域變數、方法參數，使用小駝峰式
 - 建立好習慣，有助於未來進公司、實驗室較好的適應
 - 課本內容有些不會照上面的規則(課本嘛)

C#的特性與應用

比較維度	C 語言	C++	Java	C#
程式範式	過程化 (Procedural)	多重範式 (過程化、物件導向、泛型)	強物件導向 (OOP)	多重範式 (物件導向、元件導向、函數式)
記憶體管理	手動管理 (指標、malloc/free)	手動管理 (指標、RAII、智慧指標)	自動垃圾回收 (Garbage Collection)	自動垃圾回收 (.NET GC，支援 unsafe 指標)
執行機制	直接編譯為原生機器碼	直接編譯為原生機器碼	編譯為 Bytecode，經 JVM / JIT 執行	編譯為 IL 位元組碼，經 .NET CLR / JIT 執行
跨平台性	依賴平台重新編譯	依賴平台重新編譯	極佳 ("Write Once, Run Anywhere")	極佳 (透過跨平台 .NET Core / .NET)
主要優勢	極致效能、資源佔用極低、貼近硬體	高效能與高抽象層級並存、靈活度高	生態系龐大、跨平台移植性高、穩定性高	現代化語法、開發效率高、生態系整合佳
主要應用	作業系統核心、嵌入式系統、驅動程式	遊戲引擎 (如 Unreal)、高頻交易、系統級軟體	企業級後端 (Spring)、Android 開發、大數據	企業後端 (.NET)、遊戲開發 (Unity)、Windows 應用

為什麼本課程會選用C#??

- 它是完全物件導向程式語言，符合此門課程訂義
- Visual Studio是功能最完整的 IDE 之一，且學習時免費
- 微軟視窗化軟體，易於開發桌面應用程式，維護有保障
- 具備LINQ等語法

比較維度	C# 的核心優勢	C++ 的對應痛點
記憶體管理	自動垃圾回收 (GC)：免去手動釋放記憶體，徹底杜絕記憶體洩漏 (Memory Leak) 與懸空指標 (Dangling Pointer)。 不用管，會自己處理	手動 / RAII 管理：需自行管理記憶體生命週期，稍有疏失易導致崩潰或記憶體洩漏。 但相對的，C++效率極高(手動回收)
開發生產力	極高：擁有高度整合的 .NET 生態系與豐富內建 API，軟體上線與迭代速度極快。 基於.NET的優勢	較低：需處理大量底層細節、標頭檔管理與跨平台相容性，開發與除錯週期長。
現代化語法	LINQ 與 Async/Await：內建強大的資料查詢語法 (LINQ) 與非同步處理，大幅減少冗長程式碼。 但LINQ要小心使用，可能會拖累效能	語法相對繁瑣：缺乏類似 LINQ 的原生高階查詢語法，非同步與模板語法較為複雜且難維護。
應用開發	完整框架：原生支援 WPF、MAUI、ASP.NET Core，快速打造 Web、桌面與行動端應用。	缺乏官方 UI 庫：打造桌面 UI 需依賴第三方框架（如 Qt、wxWidgets）。

比較維度	C# 的核心優勢	C++ 的對應痛點
執行安全性	極高：預設自動進行陣列邊界檢查，不允許任意指標運算，極少出現未定義行為（UB）。	風險較高：易發生指標越界、緩衝區溢位（Buffer Overflow）與未定義行為，資安漏洞風險高。 也因此能用些比較有風險，但效率較高的做法
套件與建置	統一且便利：使用 NuGet 進行套件管理，專案建置與第三方庫依賴關係簡單快速。 很方便引用第三方函式庫!!!!	建置繁雜：缺乏官方統一套件管理器，需依賴 CMake、vcpkg 等工具，標頭檔編譯極慢。 較不方便，但完成後效率與自由度極高
反射能力	強大 Reflection：可在執行階段動態檢查型別、載入組件或讀取 Attribute 標籤。	缺乏動態反射：編譯後型別資訊大多遺失，僅有受限的 RTTI，實現動態功能困難。
應用開發	完整框架：原生支援 WPF、MAUI、ASP.NET Core，快速打造 Web、桌面與行動端應用。 UI 開發很快，但要特殊風格也是要自己改	缺乏官方 UI 庫：打造桌面 UI 需依賴第三方框架（如 Qt、wxWidgets）。 Qt 很好用，但商業授權也蠻貴的...

第一章

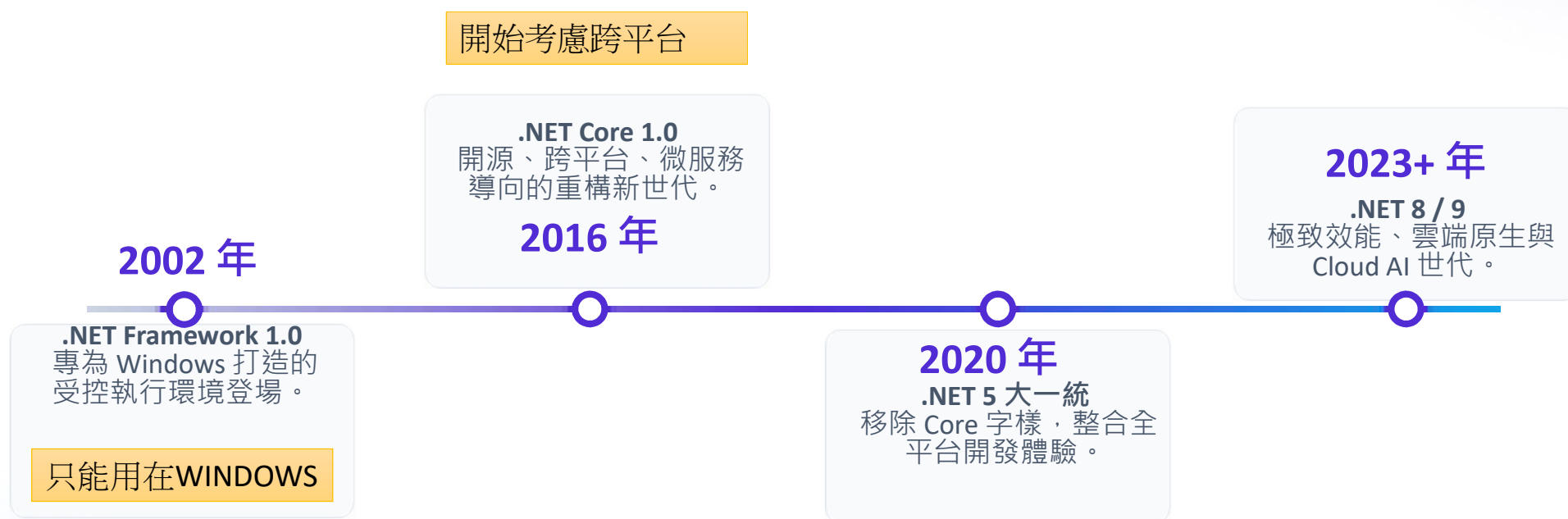
C#的發展歷史與架構

C#的第一步，決定選擇那種方案

- 執行的作業系統
 - Windows 10/11
 - Linux 指令式，UI式
- 核心選擇:決定應用作業系統，它會決定開發程式使用的核心
 - .Net 6 7 8 9 10 (可跨平台，當然要裝底層程式)
 - Framework 4.8 4.81 etc. (不可跨平台，只能用在Windows)
- 專案類型:決定應用場景，它會決定使用那種專案類型
 - **WPF (Windows Presentation Foundation)**
 - **WinForms**
 - **.NET MAUI**

.NET 生態系的關鍵演化里程碑

.Net Framework 到 .Net core 再到 .Net



變革契機：為什麼需要 .NET Core ？

▲ 傳統 Framework 的局限

隨著雲端運算與 Docker 容器化興起，傳統 .NET Framework 體積龐大、僅限 Windows 的特性成為擴展障礙。企業迫切需要更輕量、高併發的架構。

✦ .NET Core 的突破性創新

全面開源並支援跨平台（Windows, Linux, macOS）。改採模組化 NuGet 設計，支援「獨立打包部署」（Self-contained），不再依賴全域環境安裝。

.NET Framework vs 現代 .NET 對比

比較項目	.NET Framework (經典)	現代 .NET (.NET 6 / 8 / 9)
跨平台支援	僅限 Windows 平台	Windows, Linux, macOS, Android, iOS
開源狀態	閉源 / 專有軟體 (僅提供參考源碼)	100% 完全開源 (MIT 授權，由 .NET Foundation 維護)
部署機制	全域安裝於系統 (System-wide)	可隨應用程式打包獨立執行 (Self-contained)
效能表現	穩定但記憶體開銷較大	極致優化，全球頂尖的高 RPS 與低延遲表現
主導 UI 技術	WinForms, WPF, ASP.NET Web Forms	Blazor, .NET MAUI, ASP.NET Core Web API

選擇???

- 如果...，請優先選擇 **.NET Core** 。
 - 該項目需要跨平台整合。
 - 該專案需要開發微服務。
 - 依照需求選用UI設計方案。
- 如果...，請優先選擇 **.NET Framework** 。
 - 應用程式已經在.NET Framework上運行。
 - 這些應用程式需要工作流程、Web 表單或 WCF 等技術，而這些技術在 .NET Core 中並不存在。
 - 應用程式只能在Windows系統上運行。

<https://www.interviewbit.com/blog/net-core-vs-net-framework/>

比較維度	程序導向程式設計 (Procedural Programming)	物件導向程式設計 (Object-Oriented Programming, OOP)
核心概念	以「程序 / 函數 (Procedure)」為中心，著重於解決問題的步驟與執行流程。	以「物件 (Object)」為中心，將「資料 (屬性)」與「行為 (方法)」封裝在一起。
設計方法	由上而下 (Top-down)：將大任務拆解成小函數，逐層執行。	由下而上 (Bottom-up)：先建立基礎的物件與類別，再組裝成大型系統。
資料安全性	較低。資料多為全域或函數間直接傳遞，容易被意外修改或造成副作用。	較高。透過「封裝 (Encapsulation)」保護內部資料，僅透過介面存取。
擴充與重用性	較低。依賴函數的重複調用，當業務邏輯改變時，修改範圍可能牽一髮而動全身。	高。利用「繼承 (Inheritance)」與「多型 (Polymorphism)」，易於擴充且程式碼重用率高。
適用場景	小型腳本、系統底層開發、演算法實作、線性流程處理。	大型、複雜、多人協作、需長期維護與頻繁迭代的企業級軟體。
常見代表語言	C、Pascal、Fortran、BASIC	Java、C++、Python、C#、Swift

C# OOP的應用例子



C#專案類型

C# 教學進程

還有其他的功能
不在此學期課程式

第 2 階段
Class Library
OOP 封裝與架構
解耦

第 1 階段

Console App
基礎語法與控制
結構

封裝成DLL測試
模擬輸出DLL
給廠商/合作同伴使用

第 3 階段

WPF / MAUI
圖形UI與跨平台
實務

視窗程式互動
以WPF為主
小專題使用
完成完整題目

基礎指令、迴圈等
期中以前教完
TPC+考題為期中試題

Visual studio 2022

下載位置(直接查會找到2026版的):

<https://learn.microsoft.com/en-us/visualstudio/releases/2022/release-history>

Evergreen bootstrappers

The bootstrappers in the following table will always install the latest release from their respective channel.

 Expand table

Channel	Version	Channel Bootstrappers	End of Support Date
Current	17.14	Enterprise Professional Build Tools Community	January 13, 2032
Fall 2024 LTSC	17.12	Enterprise Professional Build Tools	July 14, 2026

本課程使用 Visual studio 2022

- **64 位元整合開發工具**
可使用非常大量的記憶體進行編輯、執行和偵錯，提高開發速度。
- 支援的開發底層
 - .Net 6~9
 - **.NET Framework** (4.6.2 至 4.8.1)
- **支援熱重新載入**
可在程式執行中修改程式碼即時生效，目前支持專案有 WPF, Windows Forms, ASP.NET Core, Console。
- **AI 程式碼自動完成 (IntelliCode)**
加強機器學習程式碼，使你更快編寫程式碼。
 - 這個各位同學可能沒感覺，當所有程式碼都要一個一個字KEY進去，沒有提示時...

Visual studio 2026新功能與風險

新功能

- AI 代理模式（ Copilot Agent ）
- IDE與工具鏈分離
- 效能與大型專案最佳化

風險

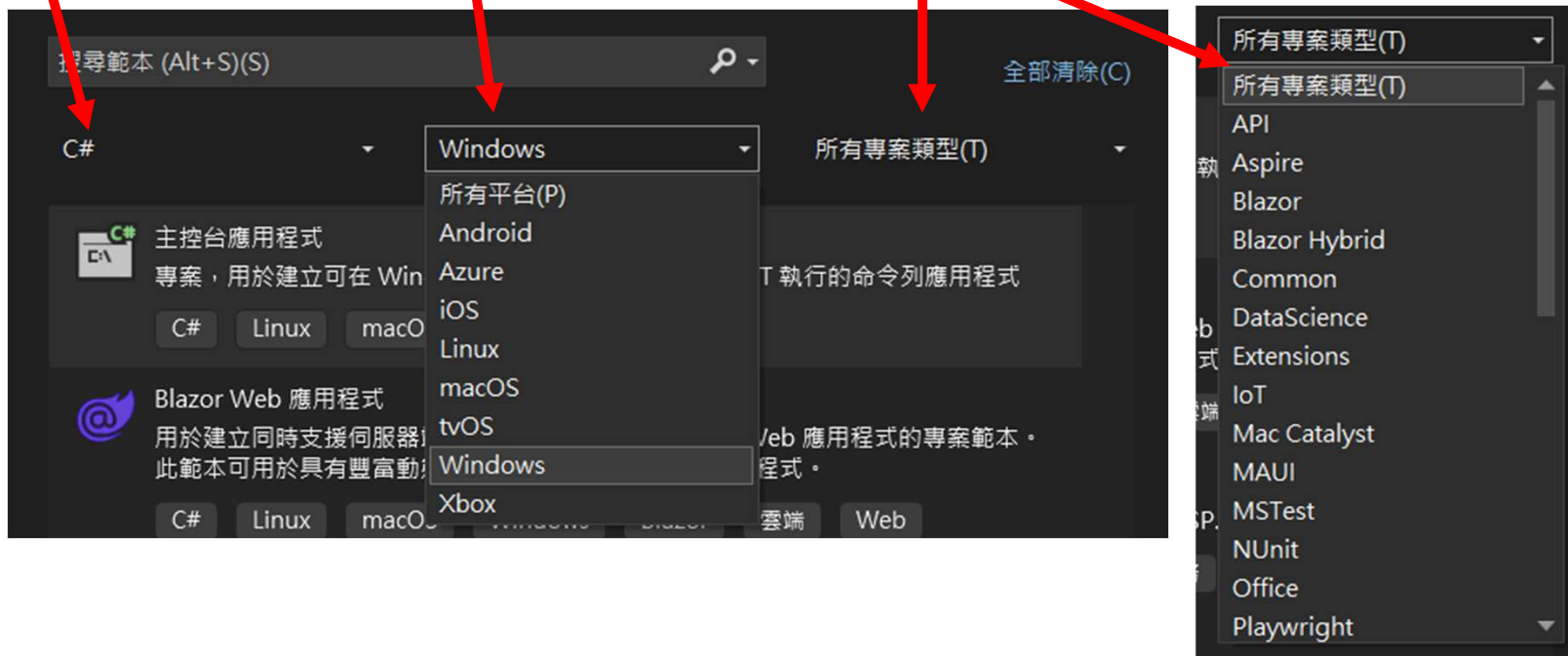
- 初期穩定性與 Bug
 - 太新的不一定好，尤其是對公司來說，穩定不出錯才是一切
- AI 成本與計算壓力
 - CPU占用率
 - RAM要求大小大增
- 功能相容性

C#使新教學 新專案

程式語言

目標系統

專案類型



C#使新教學 新專案

1. 選擇程式言語

1. 理所當然是**C#**。未來同學想學新語言，也可以試試其他語言

2. 選擇平台

1. 課程上選擇**WINDOWS**
2. 未來同學有其他專題需求，可以選擇其他系統

3. 專案類型

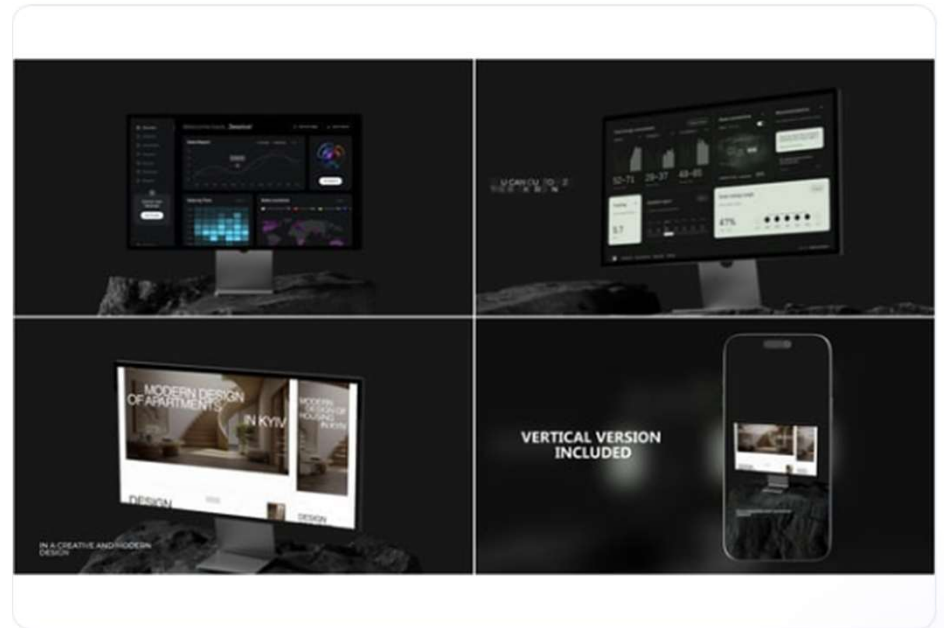
1. 主控台應用程式
2. 類別庫(DLL)
3. 視窗應用程式(見下頁)

桌面端開發：WPF vs WinForms vs MAUI

WPF (Windows Presentation Foundation): 基於 XAML 與資料綁定 (MVVM)，適合複雜企業級桌面軟體。

WinForms (Windows Forms): 拖拉式控制項、開發極速，適合小型內部工具或舊系統維護。

.NET MAUI: 現代跨平台 UI 框架，單一程式碼庫覆蓋 Windows、macOS、iOS 與 Android。



C#新建專案，該使用那個專案類型??

功能選擇 – 依照需求而定

- DLL編譯

專案類型	圖形介面 (UI)	跨平台能力	主要應用場景
Console App	無 (文字 Command)	全平台	語法學習、自動化腳本、批次處理
WPF / WinForms	Windows 原生 UI	僅限 Windows	專業桌面工具、高互動 Windows 軟體
Web API	無 (提供 JSON/XML)	全平台 / 雲端	後端微服務、手機 App 資料接口
Blazor / MVC	HTML / Web UI	瀏覽器跨平台	企業網站、全棧 C# Web 應用程式
.NET MAUI	行動與桌面原生 UI	Win/Mac/iOS/Android	跨平台行動 App 與跨系統桌面應用

課程規畫

- 主控台應用程式
 - 學習基礎語言，如判斷、迴圈、計算等
 - 依TQC+基礎物件導向語言內容為教學實例與試題
 - 建立基本程式觀念
- 視窗應用程式
 - 學習視窗控制項物件使用
 - 應用於期末繳交專題，完成完整題目

主控台程式模式

第一個程式，Hello, World!

- 測試Visual studio IDE是否正常
- 開啟一個C#專案，建立基本主控台應用程式
- 列印出內建程式最常用的輸出，"Hello, World!"
- 測試整體流程，是否正常可用

新專案

新建主控台應用程式

使用NET 8 版本(長期支援)

主控台應用程式

C#LinuxmacOSWindows主控台

專案名稱(J)

ConsoleApp3

位置(L)

D:\課程\OOP\程式測試\OOP 考試\write file

解決方案名稱(M) ⓘ

ConsoleAppTesting

☐ 將解決方案與專案置於相同目錄中(D)

其他資訊

主控台應用程式

C#LinuxmacOSWindows主控台

架構(E) ⓘ

.NET 8.0 (長期支援)

.NET 7.0 (不支援)

.NET 8.0 (長期支援)

.NET 9.0 (標準期限支援)

Linux

容器組建類型 ⓘ

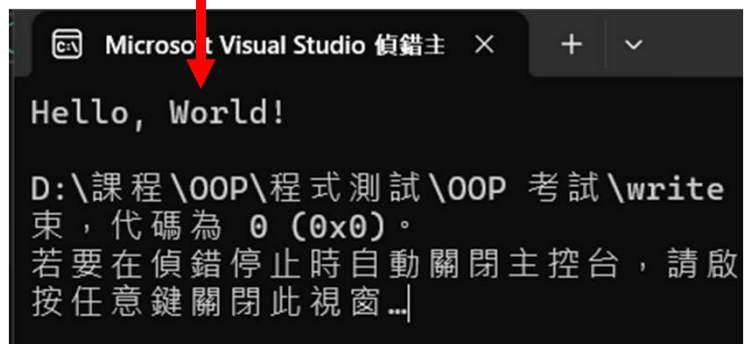
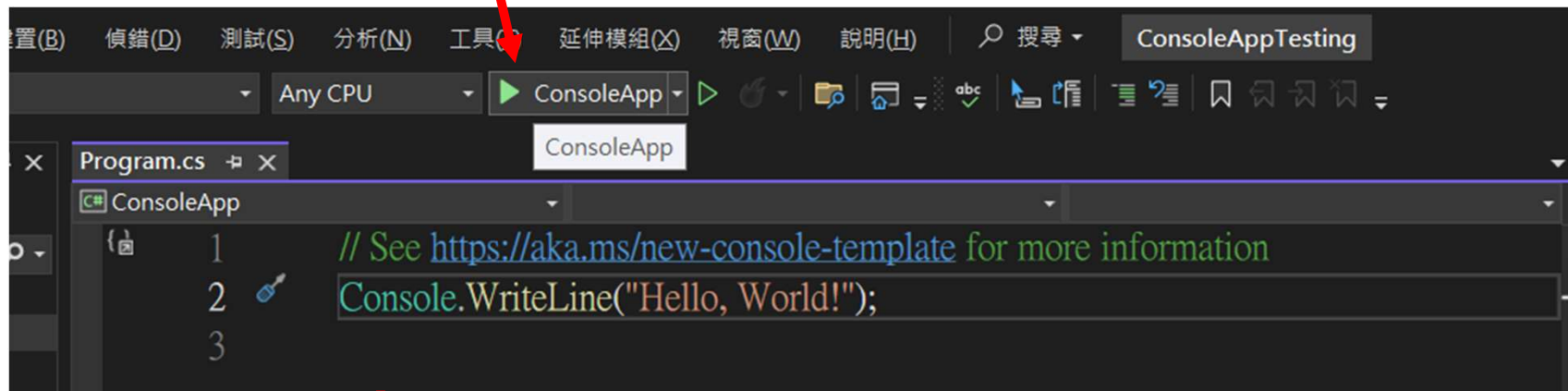
Dockerfile

☐ 不要使用最上層陳述式(I) ⓘ

☐ 啟用 native AOT 發佈 ⓘ

練習時不需要的功能:**Native AOT**（**A**head-of-**T**ime 提前編譯）是一種軟體發布技術。它在程式打包時，就把程式碼直接變成電腦看得懂的機器原生碼。這樣一來，程式不用在執行時一邊開機一邊用 **JIT**（即時編譯）翻譯，能變得更快、更小。

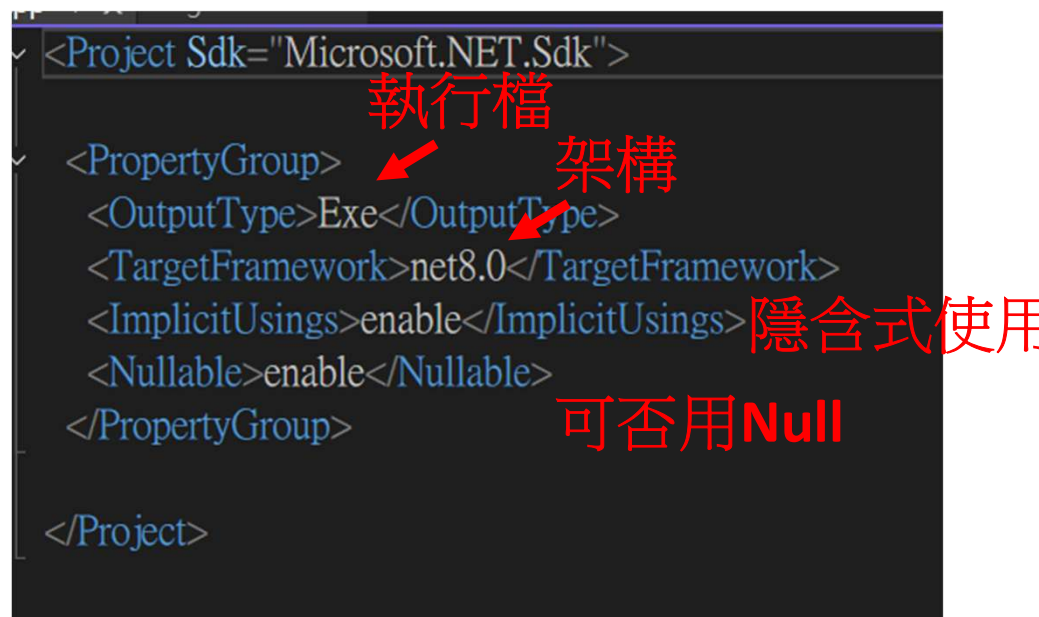
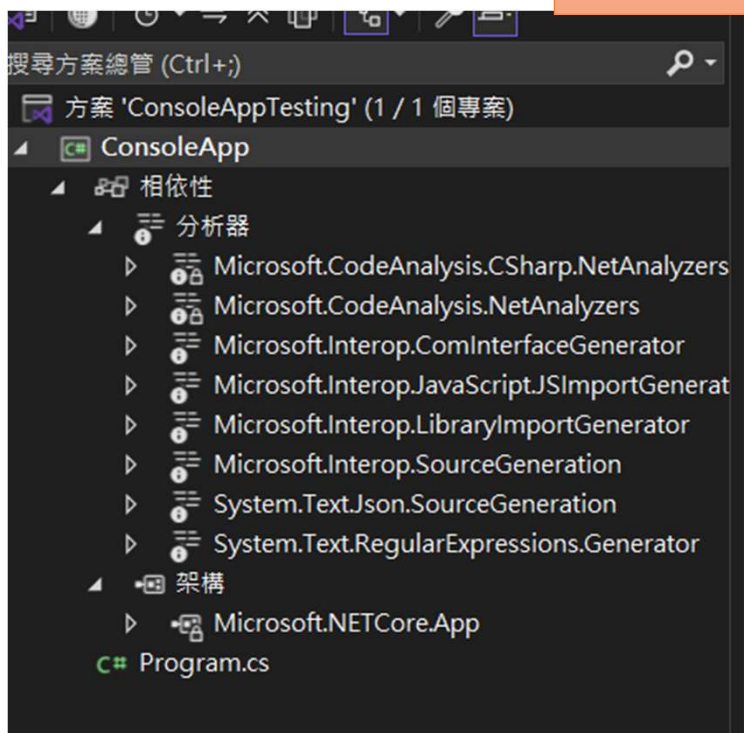
測試IDE是否正常工作



方案細節 應用程式屬性

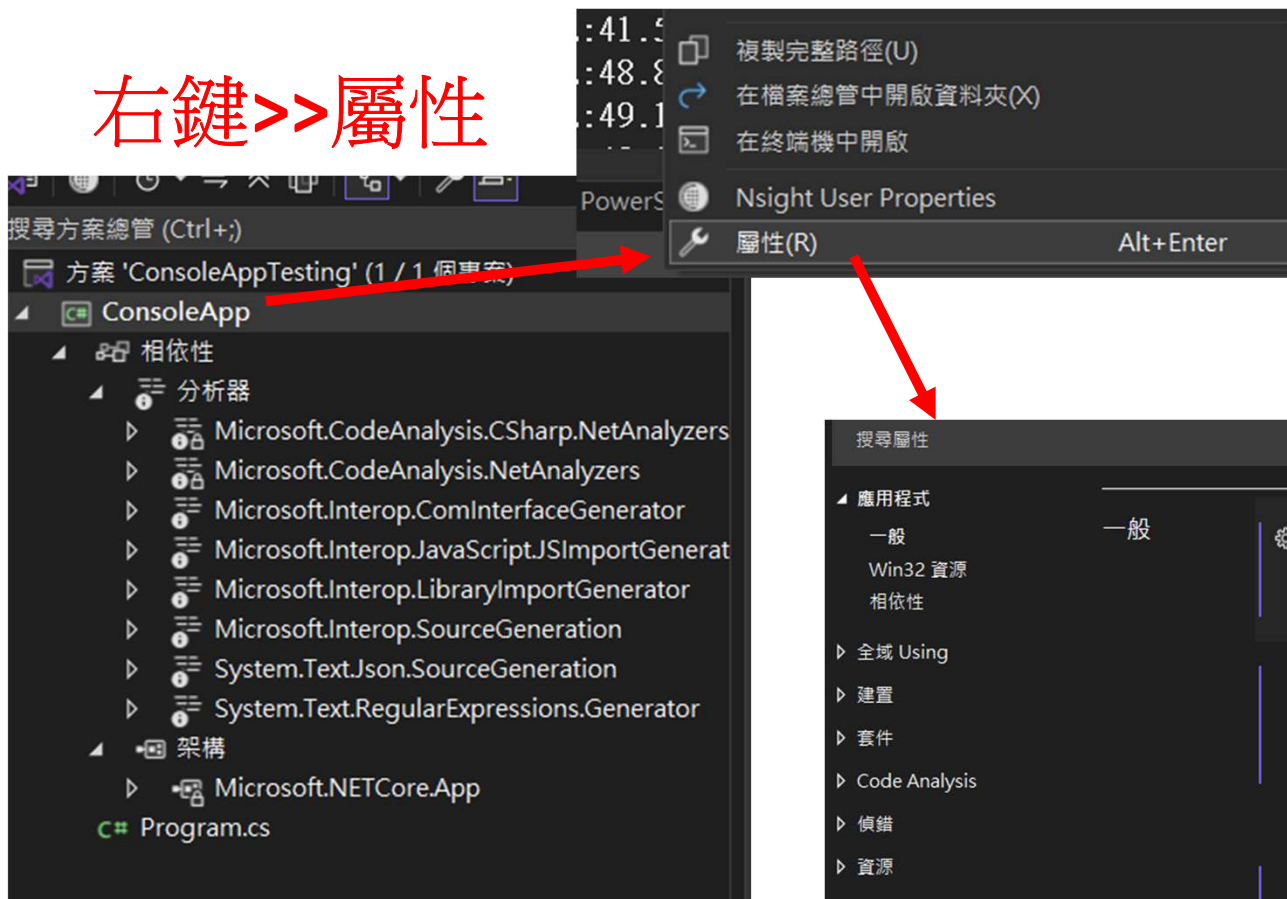
會與一開始新專案設定內容相同。

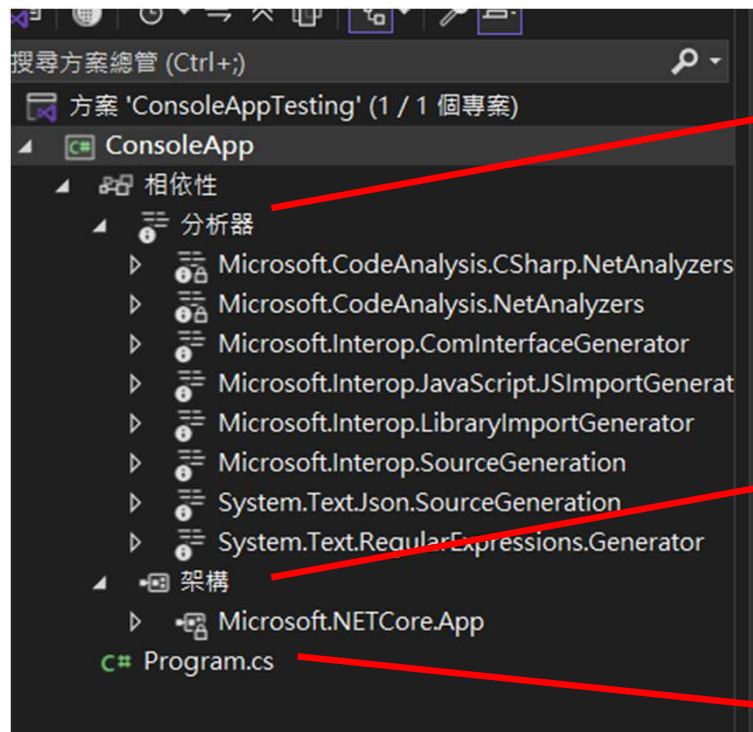
未來真的有必要修改的話.....(沒事別修改...)



最好別在這裡改!!!請見下頁

右鍵>>屬性





執行原始程式碼分析和向使用者回報問題的方法

C# 程式正在使用的核心執行環境與基本類別庫

程式本體

變數，命名原則，

型式:駝峰式

變數名稱:有義意、易於判斷的名詞

```
String inputWordForTest;  
Console.WriteLine("Hello, World!");
```

```
inputWordForTest = Console.ReadLine();
```

由主控台輸入字串，放到變數

```
Console.WriteLine(inputWordForTest);
```

變數輸出到主控台

```
Hello, World!  
1234  
1234
```

```
Console.WriteLine("Hello, World!");  
inputWordForTest = "123";  
inputWordForTest = Console.ReadLine();  
Console.WriteLine(inputWordForTest);  
Console.WriteLine(inputWordForTest);
```

綠色:警告
有潛在造成問題
的可能性
(當然,在此Case
不會出現)

紅色:錯誤,一定要修正
上方為誤寫為小寫造成語法錯誤

✖ CS0117 'Console' 未包含 'writeLine' 的定義
⚠ CS8600 正在將 Null 常值或可能的 Null 值轉換為不可為 Null 的型別。

修正:給它預設值，使其不會產生null這類無法預期的狀況。

```
inputWordForTest = Console.ReadLine() ?? "";  
Console.WriteLine(inputWordForTest);
```

<https://learn.microsoft.com/zh-tw/dotnet/api/system.console.readline?view=net-10.0>

程式語言

1-1 認識程式語言

- 程式語言的分類

- 機器語言（Machine Language）

- 是最早的程式語言，程式設計師必須查詢指令的二進位碼，並將資料轉成0 與1

- 組合語言（Assembly Language）

- 改進機器語言以二進位方式撰寫程式，提供以英文字串表達指令與資料，透過組譯器將組合語言轉成機器語言

1-1 認識程式語言

- 程式語言的分類

- 高階語言（ High-Level programming language ）

透過**編譯器**轉成機器語言，編譯器支援各類型的處理器，就可將程式轉成機器語言

也可以透過**直譯器**執行，直譯器會將程式從頭開始一行接著一行執行，透過不同處理器版本的直譯器達成跨平台

1-1 認識程式語言

- 高階語言

- 程序導向

- 以功能為出發點，可以將一個大程式分割成幾個小程式，完成這些小程式，再利用呼叫小程式組合成大程式

- 物件導向

- 以物件為出發點，將功能轉換成物件，經由操作物件達成想要的功能

C 屬於程序導向的程式語言，**Visual C#** 與**Java** 屬於物件導向的程式語言，而**C++**、**PHP** 與**Python** 屬於程序導向和物件導向的程式語言

語言轉譯程式

- 直譯器 (Interpreter)

直譯法的優點是：

1. 在直譯法下，使用者欲執行程式時，一般是執行一條命令即可。
2. 直譯程式該翻譯程式在被使用者使用過程中，一般都儲存在主記憶體，所以當使用者每次執行程式時，不必浪費取出直譯程式的 I/O 時間。
3. 由於在直譯法下，使用者不需要執行連結的工作，所以可以省去不少有關連結工作的 I/O 時間。
4. 直譯程式在執行時，是採用交談式(Interactive)的方式。亦即使用者可以很容易的與電腦作溝通之工作。

語言轉譯程式

- 編譯器 (Compiler)

欲利用編譯器來完成高階語言之翻譯然後執行時，一般需要逐次完成下列三個步驟後，才能完成使用者程式之執行工作。

1.產生目的程式

利用編譯程式將原始程式全部翻譯成機器語言程式，亦即翻譯成目的程式。

2.產生可執行之機器語言程式

利用廠商所提供的連結程式(Linker)執行連結的工作。此時才產生一個可執行的機器語言程式。該可執行的機器語言程式，在IBM PC 下稱之為可執行程式(Executable Program)，其延伸檔名(Filename Extension)一般為 EXE 或 COM。

3.執行程式

執行使用者程式，亦即執行機器語言程式。

C++與C#語法上的主要差異

語法 / 特性項目	C++	C#
程式進入點	全域函式 <code>int main()</code>	類別內的 <code>static void Main()</code> (或 C# 9+ 的 <code>Top-level statements</code>)
標頭檔與引用	使用 <code>#include <header></code> 引入標頭檔	使用 <code>using Namespace;</code> 引入命名空間
類別繼承	支援「多重類別繼承」	僅支援「單一類別繼承」，但可實現多個介面 (<code>interface</code>)
字串處理	<code>std::string</code> 或 C 風格 <code>char*</code>	原生 <code>string</code> 型別，且字串物件為不可變 (<code>Immutable</code>)
空值表示	使用 <code>nullptr</code>	使用 <code>null</code> ，值類型可使用可空語法 (如 <code>int?</code>)
控制台輸出入	<code>std::cout << "Hi"; / std::cin >> x;</code>	<code>Console.WriteLine("Hi"); / Console.ReadLine();</code>
委派與事件	無原生語法 (需使用函式指標、 <code>std::function</code> 或 <code>Lambda</code>)	提供原生 <code>delegate</code> 與 <code>event</code> 關鍵字支援事件驅動模型

```
using FellowOakDicom;  
using FellowOakDicom.Imaging;
```

```
string? studentName;  
string studentNumber = null;
```

```
inputWordForTest = Console.ReadLine();
```

Command line型程式指令
Colsole

Console 常用指令

Console.功能 .後方可以直接由文字知道此功能用途，且使用大駝峰式。符合命名原則。

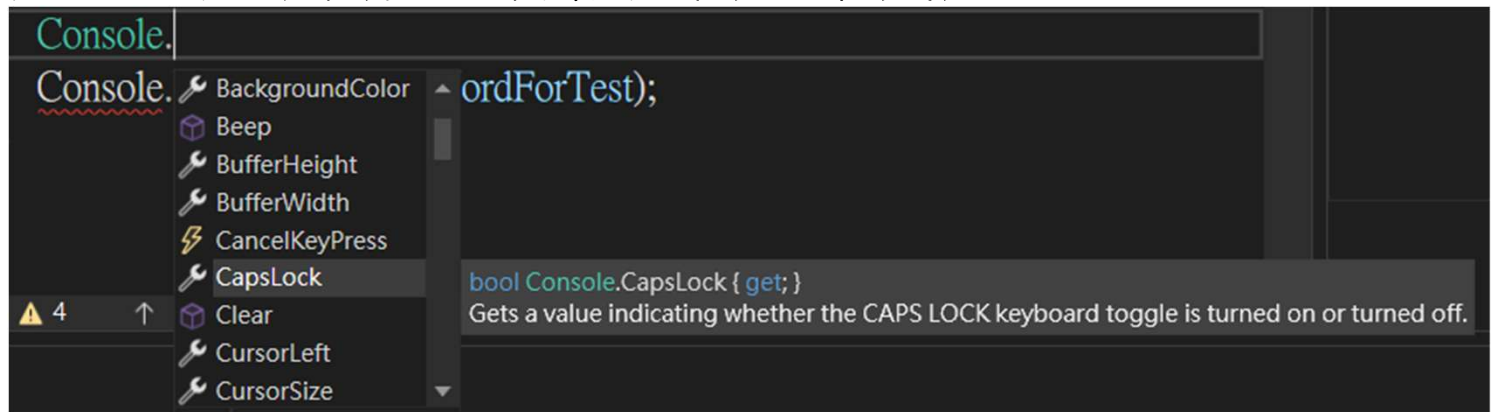
<code>Console.Write("")</code>	向控制台直接寫入字串，不進行換行，可繼續接著前面的字寫入。
<code>Console.WriteLine();</code>	表示向控制台寫入字串後換行。
<code>Console.Read();</code>	表示從控制台讀取字串，不換行。
<code>Console.ReadLine();</code>	表示從控制台讀取字串後進行換行。
<code>Console.ReadKey();</code>	取得鍵盤按下的下一個字符或功能鍵，按下的鍵顯示在控制台畫面中。
<code>Console.Beep();</code>	通過控制台播放提示音。
<code>Console.Clear();</code>	清除控制台緩衝區和相應的控制台畫面的顯示訊息。

Console 指令。續

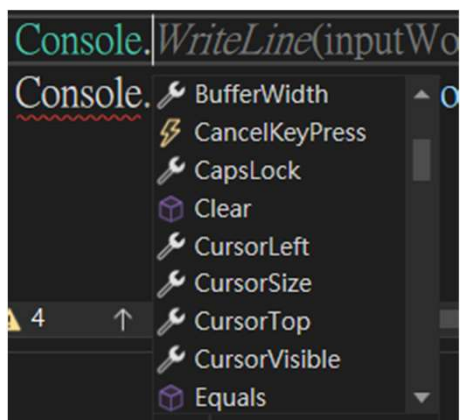
實際上，我更建議大家使用時，打入指令後仔細看一下可拉取的選項，它表示了所有可用的物件。

Mouse 指到特定項後，可以看到它的說明與應用。
初步判斷功能可由指令文字來判斷。

指令是教不完的，同學需要自行依照需求來找指令。



續



- **方塊**（通常是色塊、立方體或小方格）：代表類別（**Class**）、結構（**Struct**）或變數/欄位（**Field**）。
- **閃電**：代表事件（**Event**）（例如按鈕點擊 `Click` 或資料載入 `Load` 事件）。
- **工具**：通常指方法（**Method / 函式**）（有些版本看起來像小wrench/工具或小方塊滾動），代表可以執行的動作。

隱含轉換

Int 放到Long >>> OK

```
int largerNumber = 500000;  
long littleNumber = largerNumber;
```

Long 放到Int >>> 錯誤

```
long largerNumber = 500000;  
int littleNumber = largerNumber;
```



(區域變數) long largerNumber

CS0266: 無法將類型 'long' 隱含轉換成 'int'。已

明確轉換

明確轉換，都能接受!!!!

但出問題自負後果

一般用於明確知道數值範圍的轉換

小放到大，一定沒問題

```
int largerNumber = 500000;  
long littleNumber;  
littleNumber = (long)largerNumber;
```

大放到小，強制放的話，能執行，
但結果數值會有問題

```
long largerNumber = 5000000000000;  
int littleNumber;  
littleNumber = (int)largerNumber;
```

1783793664

字串的使用

這部份比C與C++方便很多

- 可以直接用"+"連結

```
string beginString = "Init";  
string midString = "Mid";
```

```
Console.WriteLine(beginString + midString);
```

```
InitMid
```

與C++一樣，C#亦有許多特殊的逸出字元

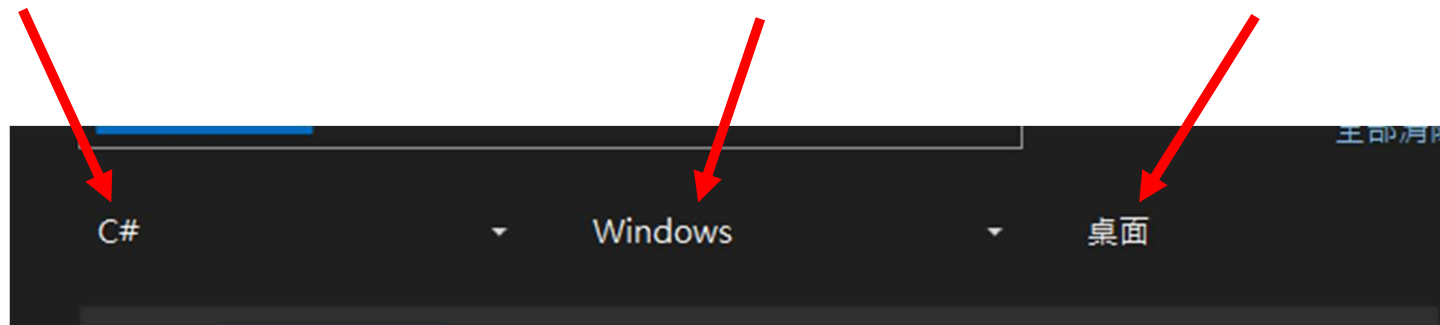
逸出序列	功能描述	程式碼範例	實際輸出結果
\"	雙引號	"\"Hello\""	"Hello"
\'	單引號	'\''	'
\\	反斜線	"C:\\Program Files"	C:\Program Files
\n	換行 (Line Feed)	"Line 1\nLine 2"	Line 1 Line 2
\r	游標歸位 (Carriage Return)	常與 \n 組合成 \r\n (Windows 系統預設換行)	-
\t	水平縮排 (Tab)	"Name:\tJohn"	Name: John
\0	Null 空字元	'\0'	(無顯示)
\b	退格 (Backspace)	"ABC\bD"	ABD

簡單練習

- 使用主控台程式模式
- 設定
 - 輸入半徑
 - 使用公式 $\text{面積} = \text{半徑} \times \text{半徑} \times \text{圓周率}$
 - 輸出，圓面積

C# windows form(Frame work類型)

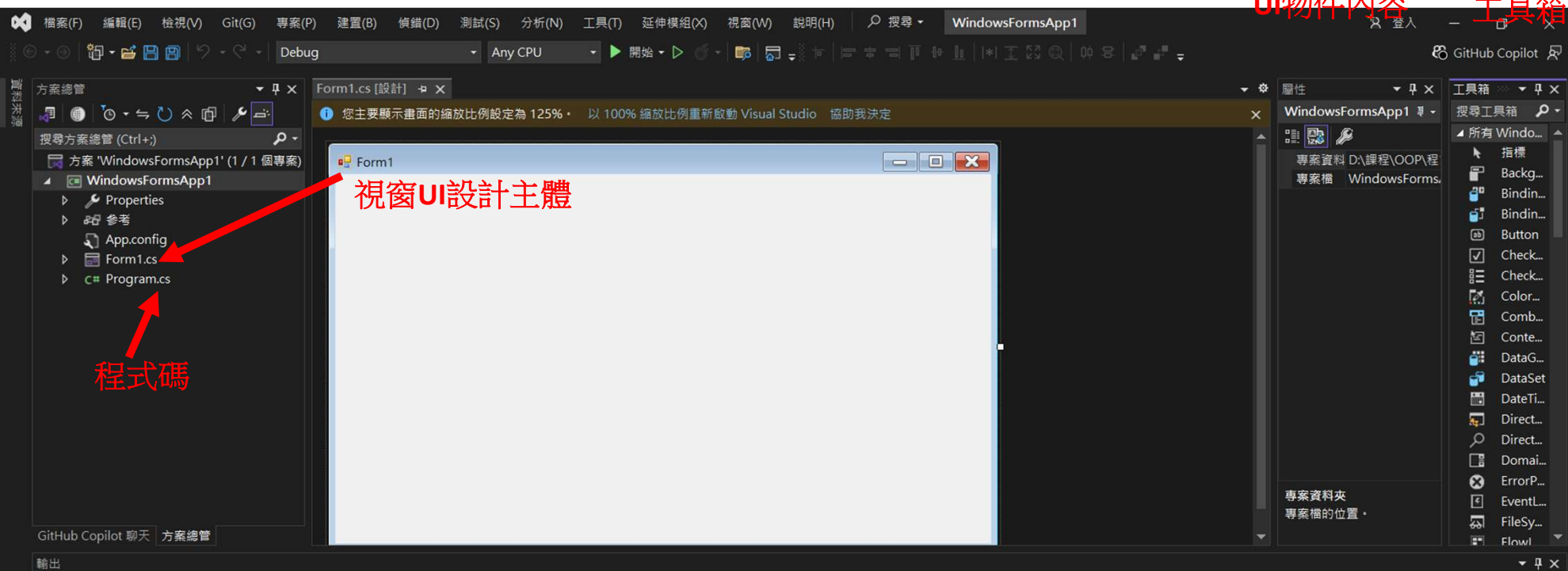
C#(程式種類) 預計執行系統 專案類型



方案總管

UI物件內容

工具箱



如果視窗和上面的不同???要自己叫出來

檢視>>>方案總管

檢視>>>工具箱

The image shows a Visual Studio interface with the 'View' menu open. The menu lists various tool windows and their keyboard shortcuts. Two red arrows point from the menu to two separate tool windows. The first arrow points from '方案總管' (Solution Explorer) to a window titled '搜尋方案總管 (Ctrl+;)'. The second arrow points from '工具箱' (Toolbox) to a window titled '工具箱'.

方案總管 (Ctrl+;) 視窗內容:

- 方案 'WindowsFormsApp1' (1 / 1 個專案)
- WindowsFormsApp1
 - Properties
 - 參考
 - App.config
 - Form1.cs
 - Program.cs

工具箱 視窗內容:

- 所有 Windows Form
 - 指標
 - BackgroundWorker
 - BindingNavigator
 - BindingSource
 - Button
 - CheckBox
 - CheckedListBox
 - ColorDialog

之後，
就可以開始測試你的第一個程式了

- 程式的“使用”沒有捷徑，只有“多練習”
 - 因為它只是一個操作方法
 - 不同IDE使用流程不同
 - 是人訂立的方法
- 程式的“設計”，則是邏輯、方塊圖、流程圖的實現
 - 這我個比較為重視，所以才採用計畫書型式做為期末報告